Experiment No: 3

Course Outcome: CO3 **Blooms Level: L3**

Aim: Data Cleaning and Storage- Preprocess, filter and store social media data for business (Using Python, MongoDB, R, etc).

Abstract:

This experiment emphasizes the preprocessing, filtering, and storage of social media data for business applications using tools such as Python, MongoDB, and R. Raw data collected from social media platforms often contains noise, missing values, duplicates, and unstructured content, which can hinder meaningful analysis. Various data cleaning techniques such as data normalization, removal of irrelevant records, handling missing values, and text preprocessing are applied to improve data quality. The cleaned data is then structured and stored efficiently in databases like MongoDB to support scalability and quick retrieval. This process ensures that social media data is reliable, well-organized, and ready for advanced analytics, enabling businesses to derive accurate insights for decision-making and strategic planning..

Sample Input and Output:

Case 1: Provide raw social media data (CSV/JSON/text) containing user comments, reviews, and metadata.
Output: Raw dataset is successfully loaded into the preprocessing environment.

Case 2: Apply data cleaning operations such as removal of duplicates, handling missing values, and text normalization using Python.
Output: Cleaned and filtered dataset with reduced noise and improved data quality is generated.

Case 3: Store the processed data into a database such as structured files (CSV/JSON) or MongoDB.
Output: Cleaned social media data is successfully stored and ready for retrieval and analysis.

Theory:

Social media data cleaning and preprocessing is a crucial step in transforming raw, unstructured data into a usable format for business analytics. Social media datasets, such as tweets, often contain noise in the form of duplicate records, missing values, special characters, emojis, URLs, hashtags, and

irrelevant text. Without proper cleaning, this noisy data can negatively impact the accuracy of analytical models and business insights.

Twitter Dataset (tweet.csv):
The tweet.csv file contains raw Twitter data including tweet text, user information, timestamps, and other metadata. Since tweets are short, informal, and dynamic in nature, they require extensive preprocessing to handle abbreviations, slang, emojis, mentions, and URLs. This dataset serves as the primary input for performing data cleaning and preprocessing operations.

Data Preprocessing Techniques:
Data preprocessing involves removing duplicate tweets, handling missing or null values, converting text to lowercase, removing URLs, hashtags, mentions, punctuation, and stop words. Additional techniques such as tokenization and stemming or lemmatization are applied to standardize textual data. These steps improve the overall quality and consistency of the dataset, making it suitable for reliable analysis and visualization.

Data Visualization:
Data visualization plays an important role in understanding patterns, trends, and relationships within social media data. After cleaning and preprocessing, visualization techniques such as bar graphs, line charts, heat maps, and word frequency plots are applied using tools like Python libraries (Matplotlib, Seaborn) or R (ggplot2). Heat maps help identify correlations between variables such as tweet activity over time, user engagement, or sentiment distribution. Graphs and charts provide a clear and intuitive representation of trends, enabling businesses to quickly interpret customer behavior, popular topics, and engagement patterns from social media data.

Data Storage:
The processed and visualized data is stored in structured formats such as CSV or JSON files, or in databases like MongoDB. MongoDB's flexible document-based structure makes it suitable for storing large volumes of social media data along with metadata and analysis results. Proper storage ensures efficient data management, retrieval, and scalability for future business intelligence and analytical tasks.
.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.stem.porter import PorterStemmer

import string
import re
import textblob
from textblob import TextBlob
import os

from wordcloud import WordCloud, STOPWORDS
from wordcloud import ImageColorGenerator
import warnings
%matplotlib inline
```

Show hidden output

```
from google.colab import files

uploaded = files.upload()
```

Show hidden output

```
import pandas as pd

# Assuming the uploaded file is 'Tweets.csv'
tweets_df = pd.read_csv('Tweets.csv')
```

```
tweets_df.head(5)
```

```
tweets_df.to_csv("airlinetweets.csv")
tweets_df.to_csv()
```

Show hidden output

```
tweets_df.shape
```

Show hidden output

```
tweets_df.head
```

Show hidden output

```
tweets_df.info()
```

Show hidden output

```
tweets_df['negativereason'].value_counts()
```

Show hidden output

```
#Heat Map for missing Values
plt.figure(figsize=(17, 5))
sns.heatmap(tweets_df.isnull(), cbar=True, yticklabels=False)
plt.xlabel("Column_Name", size=14, weight="bold")
plt.title("Places of missing values in column",size=17)
plt.show()
```

Show hidden output

```
import plotly.graph_objects as go
Top_Location_Of_tweet= tweets_df['airline'].value_counts().head (10)
```

```
print(Top_Location_Of_tweet)
```

```
from nltk. corpus import stopwords
stop = stopwords.words('english')
tweets_df['text'].apply(lambda x: [item for item in x if item not in stop])
tweets_df.shape
```

Show hidden output

```
!pip install tweet-preprocessor
```

Show hidden output

```
#Remove unnecessary characters
punct  =  ['%','/',':','\\','&amp','&',';','?']

def remove_punctuations(text):
  for punctuation in punct:
    text = text.replace(punctuation,'')
  return text
```

```
tweets_df['text'] = tweets_df['text'].apply(lambda x: remove_punctuations(x))
```

```
#Drop tweets that has empty text fields
tweets_df['text'].replace( '', np.nan, inplace=True)
tweets_df.dropna(subset=["text"],inplace=True)
len(tweets_df)
```

Show hidden output

```
tweets_df = tweets_df.reset_index(drop=True)
tweets_df.head()
```

Show hidden output

```
sns.set_style('whitegrid')
%matplotlib inline
stop=stop+['Virgin America' , 'San Francisco' , 'Boston' , 'New York',
           ' customer' ,'flight' , 'airline', 'San Diego' , 'Oakland' ,
           'California']
def plot_20_most_common_words(count_data, count_vectorizer) :
  import matplotlib. pyplot as plt
  words = count_vectorizer.get_feature_names_out()
  total_counts = np. zeros(len(words))
  for t in count_data:
    total_counts = t.toarray()[0]

  count_dict = (zip(words, total_counts))
  count_dict = sorted(count_dict, key=lambda x:x[1],reverse=True)[0:20]
  words = [w[0] for w in count_dict]
  counts = [w[1] for w in count_dict]
  x_pos = np.arange(len(words))

  plt.figure(2, (40,40))
  plt.subplot(title = '20 most common words')
  sns. set_context('notebook',font_scale=4,rc={ 'lines.linewidth' :2.5})
  sns.barplot(x=x_pos, y=counts, palette='husl')
  plt.xticks(x_pos, words, rotation=90)
  plt.xlabel('words')
  plt.ylabel('counts')
  plt.show()

count_vectorizer = CountVectorizer(stop_words=stop)
# Fit and transform the processed titles
count_data = count_vectorizer.fit_transform(tweets_df['text'])
# print(count_vectorizer)
# print(count_data)
# Visualise the 20 most common words
plot_20_most_common_words(count_data,count_vectorizer)
plt.savefig( 'saved_figure.png')
```

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer

sns.set_style('whitegrid')
%matplotlib inline

stop = stop + ['Virgin America', 'San Francisco', 'Boston', 'New York',
               'customer', 'flight', 'airline', 'San Diego', 'Oakland',
               'California']

def plot_20_most_common_words(count_data, count_vectorizer):
    words = count_vectorizer.get_feature_names_out()
    total_counts = np.zeros(len(words))

    for t in count_data:
        total_counts += t.toarray()[0]

    count_dict = dict(zip(words, total_counts))
    count_dict = sorted(count_dict.items(), key=lambda x: x[1], reverse=True)[:20]

    words = [w[0] for w in count_dict]
    counts = [w[1] for w in count_dict]

    x_pos = np.arange(len(words))

    plt.figure(figsize=(12, 6))
    sns.set_context('notebook', font_scale=1.5)
    sns.barplot(x=x_pos, y=counts, palette='husl')
    plt.title('20 most common words')
    plt.xticks(x_pos, words, rotation=45, ha='right')
    plt.xlabel('Words')
    plt.ylabel('Counts')
    plt.show()


count_vectorizer = CountVectorizer(stop_words=stop)
count_data = count_vectorizer.fit_transform(tweets_df['text'])

# Visualize the 20 most common words
plot_20_most_common_words(count_data, count_vectorizer)
```

```
import cufflinks as cf
cf.go_offline()
cf.set_config_file(offline=False, world_readable=True)

def get_top_n_bigram(corpus, n=None) :
  vec = CountVectorizer(ngram_range=(2, 4), stop_words="english").fit(corpus)
  bag_of_words = vec.transform(corpus)
  sum_words = bag_of_words.sum(axis=0)
  words_freq =[(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
  words_freq =sorted(words_freq, key = lambda x: x[1], reverse=True)
  return words_freq[:n]

common_words = get_top_n_bigram(tweets_df['text'] , 8)
mydict={}
for word, freq in common_words:
  bigram_df = pd.DataFrame(common_words,columns = ['ngram', 'count'])

bigram_df.groupby( 'ngram' ).sum()['count'].sort_values(ascending=False).sort_values().plot.barh(title =
                    'Top 8 bigrams',color='orange' , width=.4, figsize=(12,8),stacked = True)
```

Show hidden output

```
def get_subjectivity(text):
  return TextBlob(text).sentiment.subjectivity
def get_polarity(text):
  return TextBlob(text).sentiment.polarity
```

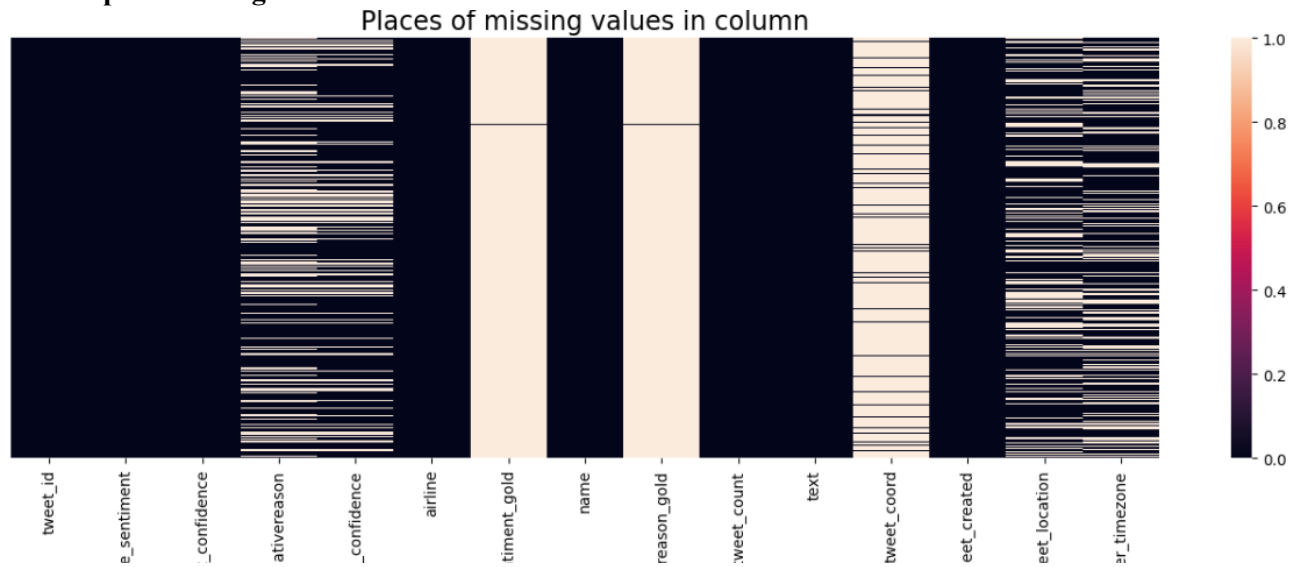
```
tweets_df['subjectivity']=tweets_df[ 'text'].apply(get_subjectivity)
tweets_df[ 'polarity' ]=tweets_df[ 'text'].apply(get_polarity)
tweets_df.head()
```

Output:

Twitter dataset column information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14640 entries, 0 to 14639
Data columns (total 15 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   tweet_id                    14640 non-null  int64
 1   airline_sentiment           14640 non-null  object
 2   airline_sentiment_confidence 14640 non-null  float64
 3   negativereason              9178 non-null   object
 4   negativereason_confidence   10522 non-null  float64
 5   airline                     14640 non-null  object
 6   airline_sentiment_gold      40 non-null     object
 7   name                        14640 non-null  object
 8   negativereason_gold         32 non-null     object
 9   retweet_count               14640 non-null  int64
 10  text                        14640 non-null  object
 11  tweet_coord                 1019 non-null   object
 12  tweet_created               14640 non-null  object
 13  tweet_location              9907 non-null   object
 14  user_timezone               9820 non-null   object
dtypes: float64(2), int64(2), object(11)
memory usage: 1.7+ MB
```

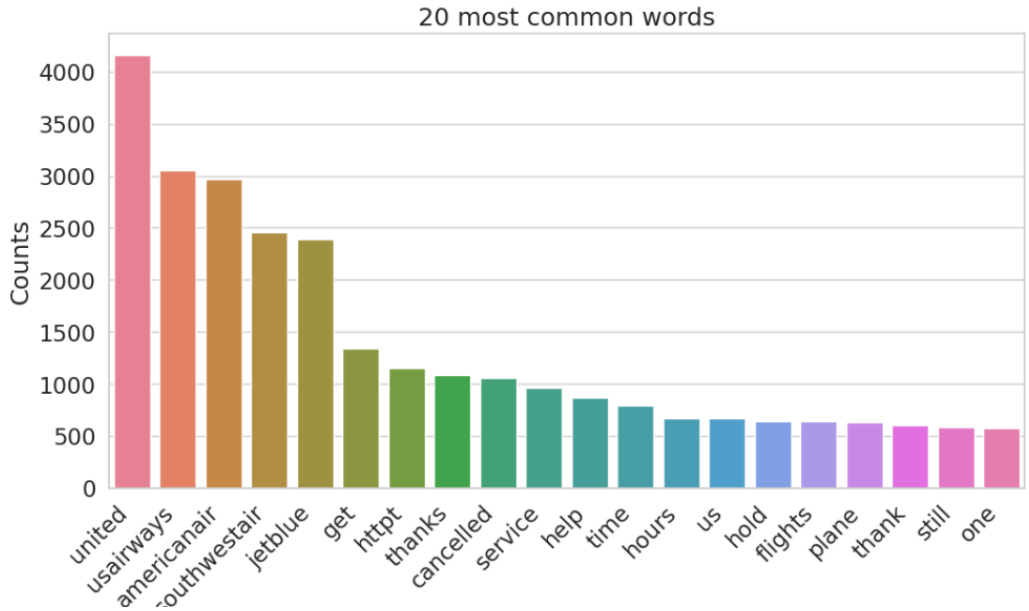
HeatMap for missing values:

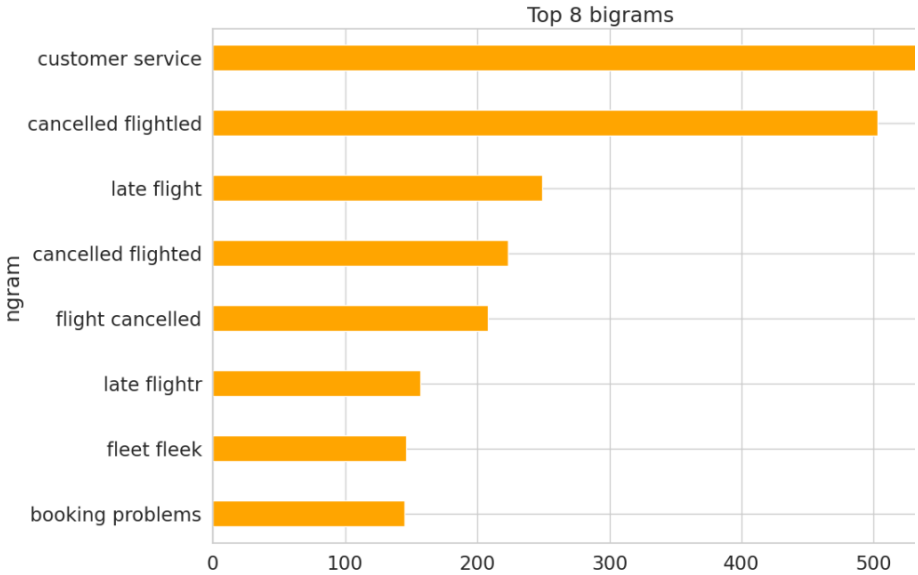


Top Location of tweet:

```
airline
United            3822
US Airways        2913
American          2759
Southwest         2420
Delta             2222
Virgin America     504
Name: count, dtype: int64
```

20 most common words:


20 most common words
Counts
4000
3500
3000
2500
2000
1500
1000
500
0
united
usairways
americanair
southwestair
jetblue
get
httpt
thanks
cancelled
service
help
time
hours
us
hold
flights
plane
thank
still
one


Top 8 bigrams
ngram
customer service
cancelled flightled
late flight
cancelled flighted
flight cancelled
late flightr
fleet fleek
booking problems
0
100
200
300
400
500

Conclusion:

In this experiment, the Tweet.csv dataset was successfully processed using data cleaning and preprocessing techniques. Noise such as duplicate records, missing values, and irrelevant text was removed to improve data quality. Visualization techniques including graphs and heat maps were applied to effectively represent patterns and trends within the data. Overall, this experiment demonstrates the importance of data cleaning, preprocessing, and visualization in transforming raw social media data into meaningful information for business analysis and decision-making.

Exercise 1:

To collect social media comments from YouTube using the YouTube Data API, store the extracted data in a structured .csv format, and perform data cleaning, preprocessing, and visualization for analyzing patterns, trends, and deriving useful business insights.

Exercise 2:
Twitter generates large volumes of textual data reflecting public discussions on various topics. Analysing this data can provide insights into commonly discussed themes and patterns.

In this exercise, you are required to collect, preprocess, and analyse Twitter data related to a selected keyword or topic using the snscrape library. The task involves scraping tweets within a specified date range and applying natural language processing (NLP) techniques to extract meaningful insights from unstructured tweet text.